



Institut TIC
de Barcelona

Llenguatges de marques i sistemes de gestió d'informació

Python

- Python (creat en 1991) és un dels llenguatges de programació més populars a l'actualitat.
- Usos comuns:
 - Data science & machine learning
 - Desenvolupament web
 - Automation & scripting
 - Intel·ligència artificial
 - Desenvolupament back-end

- Característiques:
 - Sintaxi senzilla d'aprendre
 - Gran quantitat de llibreries
 - Multiplataforma
 - Ampli recolzament de comunitat

- Python 3 vs Python 2

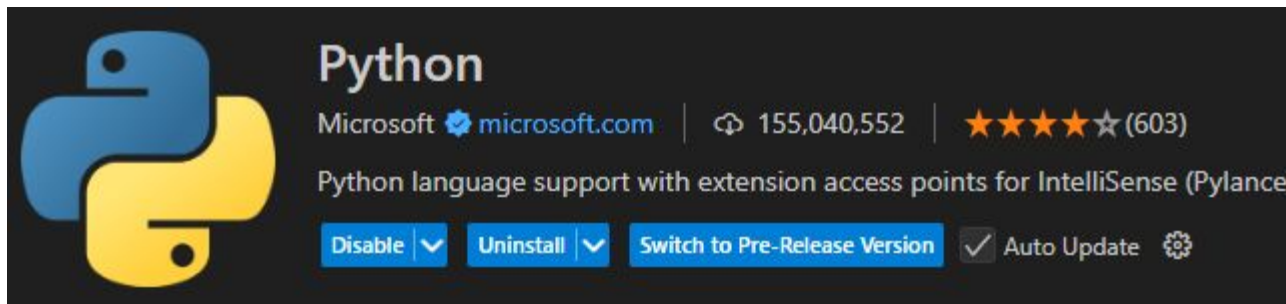
```
PS C:\Users\profe> python --version
Python 3.12.9
PS C:\Users\profe> python
Python 3.12.9 (tags/v3.12.9:fdb8142, Feb  4 2025, 15:27:58) [MSC v.1942 64 bit (AMD64)] on win32
```

```
>>> import sys
>>> print(sys.version)
3.12.9 (tags/v3.12.9:fdb8142, Feb  4 2025, 15:27:58) [MSC v.1942 64 bit (AMD64)]
```

<https://www.python.org/>

```
PS C:\Users\profe> python
Python 3.12.9 (tags/v3.12.9:fd8b142, Feb  4 2025, 15:27:58) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hola món")
Hola món
>>> exit()
PS C:\Users\profe>
```

```
PS C:\Users\profe\Dropbox\insti\xml\Python> python hola_mon.py
Hola món
PS C:\Users\profe\Dropbox\insti\xml\Python> |
```



- Sintaxi dissenyada per llegibilitat
- Fa servir salt de línia per completar una comanda (no cal ;)

```
print("Hola món")

# Això és un comentari
print("Com estàs?")

"""
Això és un comentari
de més d'una línia
perquè Python ignora
Strings no assignats
a variables
"""
```

- Identació:
 - A altres llenguatges es fa servir per llegibilitat
 - A Python la identació (amb tabuladors o espais) serveix per definir l'abast del codi en lloc de { }, begin end, ...

```
if 2<5:  
    print("Dos és menor que 5")  
  
if 2<5:  
    print("Dos és menor que 5")
```

- Identació:
 - La identació pot ser amb tabulador o espais en blanc, però aquests han de ser iguals dins del mateix bloc de codi

```
if 2<5:      #OK
|   print("Dos és menor que 5")

if 2<5:      #OK
| print("Dos és menor que 5")

if 2<5:      #OK
| | |   print("Dos és menor que 5")
```

```
if 2<5:
| print("Dos és menor que 5")
| print("Dos és menor que 5") #Malament
```


- Nom de les variables:
 - Han de començar per una lletra o _
 - Únicament pot tenir valors alfanumèrics i _
 - Són case-sensitive
 - No pot ser cap palabra clau de Python

- Variables:
 - Les variables no es declaren, es creen en el moment que li assignes un valor
 - Té el perill que si escrius malament una variable, crearà una altra
 - Case sensitive

```
x = 5  
y = "Joan"  
print(x)  
print(y)
```

```
laMevaVariable = 3  
laMeuaVariable = 9  
  
print(laMevaVariable)
```

```
nom = "Nikolai Gogol"  
Nom = "Muhammad Iqbal"
```

- Tipus de les variables:
 - Les variables no són de cap tipus en particular i, de fet, poden canviar de tipus
 - Els strings es poden definir amb cometes simples o dobles
 - Per definir una variable d'un tipus en particular fem servir *casting*
 - `type(var)` retorna el tipus d'una variable

```
# x és de tipus int  
x = 10  
# x ara és un string  
x = "Fatima"  
print(x)
```

```
nom = "Nikolai Gogol"  
nom = 'Muhammad Iqbal'
```

```
x = str(3)    # x val '3'  
y = int(3)    # y val 3  
z = float(3)  # z val 3.0
```

```
>>> print(type(y))  
<class 'str'>
```

- Assignacions múltiples

```
a, b, c = "ASIX", "DAM", "DAW"
```

```
x = y = z = "DAM"
```

```
#Si tens valors a una llista els pots "desempaquetar" en variables
```

```
fruits = ["apple", "banana", "cherry"]
```

```
x, y, z = fruits
```

```
print(x)
```

```
print(y)
```

```
print(z)
```

- Per mostrar valor de variables

```
x = "ITIC mola molt"
print(x)

x = "ITIC"
y = "mola"
z = "molt"
print(x, y, z)
print(x + y + z)

#No amb números
x = 5
y = 10
print(x + y)      #Mostra 15
print(x, y)       #Mostra 5 10
```

- Definició de Variables
 - globals: variables definides fora d'una funció
 - locals: variables definides dins de funcions

```
x = "impresionant"

def miFuncio():
    x = "una tabarra"
    print("La pel·licula és " + x)

miFuncio()

print("La pel·licula és " + x)
```

- Definició de Variables
 - globals: variables definides fora d'una funció
 - locals: variables definides dins de funcions

```
x = "impresionant"

def miFuncio():
    x = "una tabarra"
    print("La pel·licula és " + x)

miFuncio()

print("La pel·licula és " + x)
```

```
La pel·licula és una tabarra
La pel·licula és impresionant
```

- Per modificar el valor d'una variable global des d'una funció

```
x = "impresionant"

def miFuncio():
    global x
    x = "una tabarra"

miFuncio()

print("La pel·licula és " + x)
```




- Hi ha quatre tipus de dades de col·leccions
 - **Llista**: col·lecció ordenada i modificable. Permet duplicats
 - **Tupla**: col·lecció ordenada i no-modificable. Permet duplicats
 - **Conjunt** (set): col·lecció desordenada, *modificable i no indexada. No permet duplicats.
 - **Diccionari**: col·lecció ordenada i modificable. No permet duplicats

- Permeten guardar diversos items a una variable. Les llistes són:
 - **ordenades**: els elements estan en un ordre en concret i aquest no canvia. Quan s'afegeix a una llista es fa al final de la mateixa
 - **modificables**: es pot canviar, afegir i eliminar elements de la llista després d'haver-se creat
 - **permeten valors duplicats**: els elements estan indexats començant per [0] i es poden repetir

```
miLlista = ["poma", "platan", "maduixa", "poma"]  
print(miLlista)  
print(miLlista[3])  
print(len(miLlista))
```

- Per mostrar un rang dins de la llista:

```
fruites = ["poma", "platan", "maduixa", "poma", "pinya", "mango", "papaya", "durian", "taronja"]  
print(fruit[2:7])
```

Comença en 2 (inclós) i acaba al 7 (no inclós)

```
['maduixa', 'poma', 'pinya', 'mango', 'papaya']
```

Des del començament

```
print(fruit[:4])
```

```
['poma', 'platan', 'maduixa', 'poma']
```

Fins al final

```
print(fruit[2:])
```

```
['maduixa', 'poma', 'pinya', 'mango', 'papaya', 'durian', 'taronja']
```

Darrers elements

```
print(fruit[-1])
```

```
print(fruit[-2])
```

- Per recórrer una llista:

```
for x in fruites:  
    print(x)
```

- Per comprovar si un valor existeix:

```
if "mango" in fruites:  
    print("Sí, hi ha un mango a la llista de fruites")
```

- A diferència d'altres llenguatges, les llistes poden tenir elements de diferent tipus

```
list1 = ["abc", 34, True, 40.5, "ASIX"]
```

- Modificar valors de la llista

```
fruites = ["poma", "platan", "maduixa", "poma", "pinya", "mango",  
"papaya", "durian", "taronja"]
```

```
fruites[3] = "llimona"           #Modifica element 3  
fruites[1:3] = ["meló", "pera"]  #Modifica elements 1 i 2  
fruites[-1] = "mora"             #Modifica l'últim element  
fruites.append("coco")           #Afegeix al final  
fruites.remove("mango")          #Elimina l'element mango  
fruites.pop(0)                   #Elimina l'element a la posició 0  
fruites.pop()                    #Elimina el darrer l'element
```

- Definició de la funció amb def (i codi indentat)

```
def mi_funcio():  
    print("Hola, aquesta és la meua funció")  
  
mi_funcio()
```

- Arguments

```
def mi_funcio(nom, CP):  
    print("Hola " + nom + ", aquesta és la meua funció. Codi postal", CP)  
  
mi_funcio("Arnau", 46010)
```


- Retornar un valor

```
def suma(num1, num2):  
    return num1+num2  
  
print(7, "+", 9, "=", suma(7,9))
```

- If, else, else if

```
a = 200
b = 33
if b > a:
    print("b és més gran que a")
elif a == b:
    print("a i b són iguals")
else:
    print("a és més gran que b")
```

- And, or, not

```
a = 200
```

```
b = 33
```

```
c = 500
```

```
if a > b and c > a:
```

```
    print("les dues condicions són certes")
```

```
if a > b or c > a:
```

```
    print("una de les condicions és certa")
```

```
if not a > b:
```

```
    print("a NO és més gran que b")
```

```
fruites = ["poma", "pera", "albercoc"]  
for x in fruites:           #mostra totes les fruites  
    print(x)
```

```
for x in range(6):          #mostra de 0 a 5  
    print(x)
```

```
for x in range(2, 6): # Mostra de 2 a 5  
    print(x)
```

```
for x in range(2, 30, 3): # Mostra 2, 5, 8, 11, 14, 17, 20, 23, 26, 29  
    print(x)
```



- Exercicis:
 - a. Fes un funció en Python que tinga com arguments una llista de noms i un nom, i et torni si el nom es troba a la llista
 - b. Fes un funció en Python que tinga com arguments una llista de noms i un nom, i et torni en quina posició es troba el nom o -1 si no es troba

- Exercicis:
 - a. Fes un funció en Python que tinga com arguments una llista de noms i un nom, i et torni si el nom es troba a la llista

```
fruites = ["poma", "platan", "maduixa", "poma", "pinya", "mango", "papaya", "durian", "taronja"]

def trobar_fruita(llista, fruita):
    if fruita in llista:
        return True
    else:
        return False

fruita_desitjada = "pera"

result = trobar_fruita(fruitas, fruita_desitjada)

if result == True:
    print("La ", fruita_desitjada, "es troba a la llista")
else:
    print("La ", fruita_desitjada, "NO es troba a la llista")
```

- Exercicis:
 - a. Fes un funció en Python que tinga com arguments una llista de noms i un nom, i et torni si el nom es troba a la llista

```
def trobar_fruita(llista, fruita):  
    if fruita in llista:  
        return True  
    else:  
        return False  
  
fruita_desitjada = "pera"  
  
result = trobar_fruita(fruites, fruita_desitjada)  
  
if result == True:  
    print("La ", fruita_desitjada, "es troba a la llista")  
else:  
    print("La ", fruita_desitjada, "NO es troba a la llista")
```

```
def trobar_fruita(llista, fruita):  
    return fruita in llista  
  
fruita_desitjada = "pera"  
  
if trobar_fruita(fruites, fruita_desitjada):  
    print("La ", fruita_desitjada, "es troba a la llista")  
else:  
    print("La ", fruita_desitjada, "NO es troba a la llista")
```


- Exercicis: b) Fes un funció en Python que tinga com arguments una llista de noms i un nom, i et torni en quina posició es troba el nom o -1 si no es troba

```
def posicio_nom(llista, nom):  
    for i in range(len(llista)):  
        if llista[i] == nom:  
            return i  
    return -1  
  
noms = ["Joan", "Li", "Rashid", "Kameel", "Pierre", "Joao", "Bohdana", "Roxana"]  
nom = "Roxana"  
  
nom = "Roxana"  
pos = posicio_nom(noms, nom)  
  
if pos == -1:  
    print(nom, "no es troba a la llista")  
else:  
    print(nom, "es troba a la posicio", pos)
```

- Exercicis: b) Fes un funció en Python que tinga com arguments una llista de noms i un nom, i et torni en quina posició es troba el nom o -1 si no es troba

```
def posicio_nom(llista, nom):  
    for i in range(len(llista)):  
        if llista[i] == nom:  
            return i  
    return -1
```

```
def posicio_nom2(llista, nom):  
    i = 0  
    while i < len(llista):  
        if llista[i] == nom:  
            return i  
        i = i+1  
    return -1
```

- Exercicis: b) Fes un funció en Python que tinga com arguments una llista de noms i un nom, i et torni en quina posició es troba el nom o -1 si no es troba

```
def posicio_nom(llista, nom):  
    for i in range(len(llista)):  
        if llista[i] == nom:  
            return i  
    return -1
```

```
def posicio_nom2(llista, nom):  
    i = 0  
    while i < len(llista):  
        if llista[i] == nom:  
            return i  
        i = i+1  
    return -1
```

```
def posicio_nom3(llista: list, nom: str):  
    if nom in llista:  
        return llista.index(nom)  
    else:  
        return -1
```

- Els mòduls són llibreries de codi (un fitxer que té funcions que voleu fer servir a la vostra aplicació)
- Podeu crear un mòdul simplement salvant el codi en un fitxer amb extensió .py
- Per importar un mòdul, feu servir una sentència `import`.
- Quan es fa servir una funció del mòdul es fa servir la sintaxi:
`nom_modul.funcio()`
- Els mòduls poden contenir funcions o variables de qualsevol tipus

```
import llista_noms

noms = ["Joan", "Li", "Kameel", "Pierre", "Joao", "Bohdana", "Roxana"]
nom = "Roxana"

pos = llista_noms.posicio_nom3(noms, nom)

if pos == -1:
    print(nom, "no es troba a la llista")
else:
    print(nom, "es troba a la posicio", pos)
```

- Exercici: separeu el codi de dels exercicis anteriors de funcions i llistes en dos fitxers:
 - un fitxer contenint la definició de les funcions
 - un altre fitxer que importa el fitxer anterior en forma de mòdul i fa crides a les funcions

- Podeu renombrar (ficar un alias) a un mòdul per fer servir una paraula més convenient al vostre codi fent servir **as** :

```
import el_meu_fitxer_de_modul_de_llistes as llista

noms = ["Joan", "Li", "Rashid", "Kameel", "Pierre", "Joao", "Bohdana", "Roxana"]
nom = "Roxana"

pos = llista.posicio_nom3(noms, nom)

if pos == -1:
    print(nom, "no es troba a la llista")
else:
    print(nom, "es troba a la posicio", pos)
```

- Existeixen una sèrie de mòduls predefinits en Python que podeu importar en qualsevol moment.

```
import platform
import datetime
```

```
print(platform.system())
print(platform.architecture())
print(platform.python_version())
print(platform.uname())
```

```
print(datetime.date.today())
```

```
x = dir(platform)          #Mostra totes les funcions i classes del mòdul
print(x)
```


- També podeu importar únicament la part que us interressi d'un mòdul.
- Quan importeu amb **from** no cal servir el nom del mòdul quan es crida als elements del mòdul

```
from llistanoms import posicio_nom
```

```
noms = ["Joan", "Li", "Rashid", "Kameel", "Pierre", "Joao", "Bohdana", "Roxana"]  
nom = "Roxana"
```

```
pos = posicio_nom(noms, nom)
```

```
#No cal especificar el nom del mòdul  
#pos = llistanoms.posicio_nom(noms, nom)
```



- El **diccionari** és una col·lecció de dades:
 - ordenada
 - modificable
 - que no permet duplicats
- Es fa servir per guardar informació en forma de:

clau : valor

- El **diccionari** és una col·lecció de dades:
 - *ordenada (a Python 3.7 els diccionaris són ordenats. En 3.6 i versions anterior són desordenats)
 - modificable: podem afegir, eliminar i modificar elements del diccionari després de crear-lo
 - que no permet duplicats: els diccionaris no poden tenir dos valors amb la mateixa clau
- Es fa servir per guardar informació en forma de:

`clau : valor`

```
miDiccionari = {  
    "nom": "The Dark Side of the Moon",  
    "autor" : "Pink Floyd",  
    "any": 1973,  
    "genere": "Rock",  
    "estudi": "Abbey Road"  
}  
print(miDiccionari)  
print(miDiccionari["nom"])      #Mostra el valor associat a  
una clau  
print(len(miDiccionari))        #Mostra la longitud
```

- Els valors del diccionari poden ser de qualsevol tipus:

```
miDiccionari = {  
    "nom": "The Dark Side of the Moon",  
    "autor" : "Pink Floyd",  
    "any": 1973,  
    "genere": "Rock",  
    "estudi": "Abbey Road",  
    "credits" : ["Roger Waters", "David Gilmore", "Nick Mason", "Richar Wright"],  
    "platinum": True  
}
```

- Com **accedir** als elements d'un diccionari

```
print(miDiccionari["nom"])      #Mostra el valor associat a una clau
print(miDiccionari.get("nom"))

print(miDiccionari.keys())      #Mostra totes les claus
print(miDiccionari.values())    #Mostra totes els valors
print(miDiccionari.items())     #Mostra totes els elements

if "genere" in miDiccionari:    #Per comprovar si una clau existeix al diccionari
    print("Sí, genere és una clau del diccionari")
```

- Com **modificar** els elements d'un diccionari

```
#Per modificar valors del diccionari  
miDiccionari["genere"] = "Pop/Rock"  
miDiccionari.update({"any" : 1974})
```

```
#Afegir elements  
miDiccionari["format"] = "cd"           #Si no existeix la clau, la crea
```

```
#Fa el mateix, modifica l'existen i si no existeix el crea  
miDiccionari.update({"format": "LP"})
```


- Com **eliminar** elements d'un diccionari

```
#Eliminar elements
```

```
miDiccionari.pop("any")
```

```
del miDiccionari["any"]
```

```
del miDiccionari
```

```
miDiccionari.clear()
```

```
#Elimina l'element del diccionari amb aquesta clau
```

```
#Fa el mateix
```

```
#Elimina el diccionari completament
```

```
#Buida un diccionari de contingut
```

- Com **recorrer** els elements d'un diccionari

```
#Mostrar totes les claus una a una
```

```
for i in miDiccionari:  
    print (i)
```

```
for i in miDiccionari.keys():  
    print(i)
```

```
#Mostra tots els valors un a un
```

```
for i in miDiccionari:  
    print(miDiccionari[i])
```

```
for i in miDiccionari.values():  
    print(i)
```

```
#Mostra claus i valors un a un
```

```
for i, j in miDiccionari.items():  
    print(i,j)
```

Method	Description
<u>clear()</u>	Removes all the elements from the dictionary
<u>copy()</u>	Returns a copy of the dictionary
<u>fromkeys()</u>	Returns a dictionary with the specified keys and value
<u>get()</u>	Returns the value of the specified key
<u>items()</u>	Returns a list containing a tuple for each key value pair
<u>keys()</u>	Returns a list containing the dictionary's keys
<u>pop()</u>	Removes the element with the specified key
<u>popitem()</u>	Removes the last inserted key-value pair
<u>setdefault()</u>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<u>update()</u>	Updates the dictionary with the specified key-value pairs
<u>values()</u>	Returns a list of all the values in the dictionary



Exercici: crea un programa per gestió de diccionaris. El programa tindrà un menú interactiu amb les següents opcions:

1. Mostrar diccionari (mostrant les tuples clau:valor en diferents línies)
 2. Afegir entrades(en format clau:valor) *Únicament s'afegiran entrades String
 3. Mostrar entrada (donada una clau)
 4. Esborrar entrada (donada una clau)
 5. Modificar entrada (donats clau:valor)
-
0. Sortir

- Python inclou un mòdul per treballar amb dades JSON
- Generalment treballarem convertint dades entre JSON i diccionaris

string amb JSON > diccionari Python

- Podem convertir una cadena amb el contingut JSON amb el mètode `json.loads()`
- El resultat serà un diccionari de Python

```
import json

# String amb dades JSON:
cadena = '{ "nom":"Joan", "cognom":"Rivau", "curs":"DAW" }'

# analitza x i crea un diccionari
dic = json.loads(cadena)
print(dic["curs"])
```

diccionari Python > string amb JSON

- Podem convertir una diccionari a JSON amb `json.dumps()`
- El resultat serà un string

```
import json

# un diccionari de Python
dic = {
    "marca": "Fender",
    "model": "Stratocaster",
    "preu": 1200
}

# el convertim a JSON
j = json.dumps(dic)

# el resultat és un string
print(j)
print(type(j))
```


- També podem convertir altres objectes de Python a JSON amb les següents equivalències:

Python	JSON
dict	Objecte
list	Array
tuple	Array
str	String
int	Number
float	Number
True	true
False	false
None	null

```
import json

x = {
    "name": "John",
    "age": 30,
    "married": True,
    "divorced": False,
    "children": ("Ann", "Billy"),
    "pets": None,
    "cars": [
        {"model": "BMW 230", "mpg": 27.5},
        {"model": "Ford Edge", "mpg": 24.1}
    ]
}

print(json.dumps(x, indent=4))
```

```
{
    "name": "John",
    "age": 30,
    "married": true,
    "divorced": false,
    "children": [
        "Ann",
        "Billy"
    ],
    "pets": null,
    "cars": [
        {
            "model": "BMW 230",
            "mpg": 27.5
        },
        {
            "model": "Ford Edge",
            "mpg": 24.1
        }
    ]
}
```

- Podem aconseguir persistència al nostre programa desant les dades a un fitxer per llegir-ho més endavant
- La funció per obrir fitxers a Python és

`open(nom_fitxer, mode)`

- On el mode indicat la forma en que es treballarà amb el fitxer:

- “r” (read) per **lectura**. Dona error si el fitxer no existeix.
- “a” (append) per afegir al final. Si el fitxer no existeix, el crea.
- “w” (write) per escriure. Si el fitxer no existeix, el crea.
- “x” per crear un fitxer. Dona error si ja existeix.

A més, es pot especificar si el fitxer es tractarà en binari (“b”) o text (“t”).

```
f = open("nom_fitxer.txt", "rt")
```

Per escriure un diccionari a fitxer:

```
#obrim el fitxer per escriure  
fitxer = open ("diccionari.json", "wt")
```

```
#Passem el diccionari a string  
stringJSON = json.dumps(dic)
```

```
#Ho escrivim al fitxer  
fitxer.write(stringJSON)
```

```
#i tanquem  
fitxer.close()
```

Per llegir un fitxer JSON a un diccionari:

```
#Obrim fitxer per lectura
fitxer = open (nomFile, "rt")

#Llegim el fitxer a un string
s = fitxer.read()

#El carreguem a un diccionari
dic = json.loads(s)

#i tanquem
fitxer.close()
```

Els decorators és una forma de modificar la funcionalitat d'una funció, embolicant-la dins d'una altra.

La funció externa rep el nom de ***decorator***, que rep la funció original com argument i retorna una versió modificada de la mateixa

Per entendre els decorators, s'han d'entendre una conceptes claus sobre les funcions Python:

- Funcions imbricades (“anidadas”)
- Passar funcions com arguments
- Retornar una funció com a valor

S'ha de recordar que a Python tot és un objecte, fins i tot les funcions.

Per exemple, podem incloure una funció dins d'una altra i retornar-la com un objecte

Funcions imbricades:

```
def externa(x):  
    def interna(y):  
        return x + y  
    return interna  
  
afegeix_cinc = externa(5)  
result = afegeix_cinc(6)  
print(result)  
  
#Resultat?
```



Passar functions com arguments:

```
def afegir(a, b):  
    return a + b
```

```
def multiplicar(a, b):  
    return a * b
```

```
def calcular(funcio, x, y):  
    return funcio(x, y)
```

```
result = calcular(afegir, 4, 6)  
print(result) # Mostra?
```

```
result = calcular(multiplicar, 4, 6)  
print(result) # Mostra?
```

Retornar una funció com a valor:

```
def salutacions(nom):  
    def hola():  
        return "Hola, " + nom + "!"  
    return hola  
  
missatge = salutacions("Johan")  
print(missatge())  # mostra "Hola, Johan!"
```

Un decorator és una funció que agafa una funció i la retorna afegint funcionalitat extra

```
def fes_maca(func):  
    def interna():  
        print("M'han decorat")    #afegeix comportament extra a la funció normal  
        func()                    #crida a la funcio original  
    return interna  
  
def funcio_normal():  
    print("Sóc normal")  
  
funcio_normal()  
  
# creem una versió decorada de la funció normal  
funcio_decorada = fes_maca(funcio_normal)  
funcio_decorada()    # cridem a la funció decorada
```

@ símbol

```
def fes_maca(func):  
    def interna():  
        print("M'han decorat")  
        func()  
    return interna  
  
#Serveix per indicar que s'afegeix funcionalitat extra  
#a la funció que estàs definint (funcio_normal)  
@fes_maca  
def funcio_normal():  
    print("Sóc normal")  
  
funcio_normal()
```

FastAPI és un marc web modern i ràpid per crear APIs amb Python.

Per instal·lar-ho:

```
pip install "fastapi[standard]"
```

```
from typing import Union
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
def read_root():
    return {"Hello": "World"}
```

```
@app.get("/items/{item_id}")
```

```
def read_item(item_id: int, q: Union[str, None] = None):
    return {"item_id": item_id, "q": q}
```


Executem el servidor:

```
$ fastapi dev main.py
```

i tindrem un servidor web al localhost:8000 que admet peticions a través del cURL enviat amb paràmetres. Ex:

```
http://127.0.0.1:8000
```

```
http://127.0.0.1:8000/items/10
```

```
http://127.0.0.1:8000/items/10?q=consulta
```

Ja heu creat una API que:

- Rep sol·licituds HTTP als paths “/” i “/items/{item_id}”.
- Ambdós camins fan operacions GET (HTTP).
- El path /items/{item_id} té un paràmetre *item_id* que hauria de ser un int.
- El path /items/{item_id} té un paràmetre de consulta opcional q de tipus string.

Exercici:

- Feu un servidor que retornarà dades de la llista d'empleats de l'empresa

```
empleats = [  
    {"id":1, "nom": "Oriol", "sou": 2000},  
    {"id":2, "nom": "Johan", "sou": 1800},  
    {"id":3, "nom": "Fatima", "sou": 3000},  
    {"id":4, "nom": "Wayan", "sou": 1700},  
    {"id":5, "nom": "Kadek", "sou": 1500},  
    {"id":6, "nom": "Nyoman", "sou": 2800},  
    {"id":7, "nom": "Ketut", "sou": 2300},  
]
```

- El servidor tornarà diferents valors en funció del path:
 - “/”: “Servei nòmnes”
 - “/empleats”: un llistat de tots els empleats a la llista
 - “/empleat/{id}”: mostra les dades de l’empleat amb id indicat

- Afegir elements

```
from pydantic import BaseModel
```

```
[...]
```

```
class Empleat(BaseModel):
```

```
    id: int
    nom: str
    sou: int
```

```
[...]
```

```
@app.post("/empleat/")
```

```
def crearUsuari(nou: Empleat):          #nou és un objecte de la classe Empleat
```

```
    #Crear un diccionari a partir del objecte "nou"
```

```
    XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

```
    #Afegir aquest diccionari al final de la llista
```

```
    XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

```
    return {"L'empleat s'ha inserit correctament"}
```

- Esborrar elements

```
@app.delete("/empleat/{id}")  
def esborrarEmpleat(id: int):          #Esborrar de la llista l'empleat amb id  
    for xxxxxxxxxxxxxxxxxxxxxxxx:  
        if (xxxxxxxxxxxxxxxxxxxxxxxx):  
            xxxxxxxxxxxxxxxxxxxxxxxx  
            return {"L'usuari s'ha esborrat correctament"}  
  
    return {"L'empleat no existeix"}
```

Exercici:

- Amplia el teu servidor per poder afegir i esborrar empleats

]